

Authentication in VicharHub (Notion Clone)

This document describes how sign-in works in this repo: email/password, Google OAuth, JWT access tokens, HttpOnly refresh cookies, and how the Cloudflare Worker protects API routes.

Overview

Auth is **not** handled by a third-party identity product (Auth0, Clerk, NextAuth). It is custom:

Layer	Role
React UI	Login/signup screen; Google redirect button
Zustand store (<code>useAuthStore</code>)	Session state, login/signup/logout, boot-time restore
<code>api()</code> helper	Attaches <code>Authorization: Bearer</code> and refreshes on 401
Cloudflare Worker (<code>worker/index.ts</code>)	Password hashing, JWTs, cookies, Google OAuth, D1 storage
Cloudflare D1	<code>users</code> and <code>refresh_tokens</code> tables

The app is a **gated SPA**: until the user is signed in, App only shows `AuthScreen`. After sign-in, vaults and pages are loaded from `/api/*` and each vault is scoped to that user.

Token model

Two credentials are issued after a successful sign-in.

Access token (JWT)

- **Lifetime**: 15 minutes (`ACCESS_TTL_SECONDS = 60 * 15`).
- **Storage**: `localStorage` key `vh_access_token` .
- **Transport**: `Authorization: Bearer <jwt>` on API calls.
- **Algorithm**: HMAC-SHA256 (`HS256`), signed with Worker secret `JWT_SECRET` .
- **Payload**: `{ sub: userId, iat, exp }` .
- **Verification**: Worker checks signature and `exp` . Then it loads the user row from D1 (`SELECT id, email, name FROM users WHERE id = ?`).

The JWT is implemented in the Worker with Web Crypto (no `jose` / `jsonwebtoken` dependency).

Refresh token

- **Lifetime**: 30 days (`REFRESH_TTL_SECONDS = 60 * 60 * 24 * 30`).

- **Storage:** HttpOnly cookie named `vh_refresh` (`SameSite=Lax` , `Path=/` , `Secure` except on localhost / 127.0.0.1).
- **Value:** 32 random bytes, base64-encoded. **Only the SHA-256 hash** of that value is stored in `D1 (refresh_tokens.id)`.
- **JavaScript cannot read this cookie.** The browser sends it automatically when `credentials: "include"` is set.

On every successful refresh, the old refresh token is **revoked** and a new one is issued (rotation). A stolen refresh token can be used at most once.

Password hashing

Email/password accounts store `password_hash` as:

```
pbkdf2:<iterations>:<salt_b64>:<hash_b64>
```

- Algorithm: PBKDF2-SHA256
- Iterations: 100,000
- Salt: 16 random bytes
- Derived key: 256 bits
- Compare: constant-time XOR of hash bytes

Google-only accounts insert an empty `password_hash ( '' )`. Email/password login against those rows fails `verifyPassword`.

Database

`users` (after `migrations/0003_google_auth.sql`)

Column	Notes
<code>id</code>	UUID (TEXT PK)
<code>email</code>	Unique, <code>NOCASE</code>
<code>password_hash</code>	PBKDF2 string, or empty for Google-only
<code>name</code>	Display name
<code>google_sub</code>	Google subject; unique when set
<code>avatar</code>	Google profile picture URL
<code>created_at</code>	Unix epoch

`refresh_tokens` (`migrations/0002_refresh_tokens.sql`)

Column	Notes
<code>id</code>	SHA-256 hex of the raw cookie token
<code>user_id</code>	FK → <code>users.id</code> ON DELETE CASCADE
<code>expires_at</code>	Unix epoch
<code>created_at</code>	Unix epoch

Migration `0001_init.sql` also created a `sessions` table. **Current auth does not use `sessions`**. Sessions are the refresh-token rows plus JWTs.

Vaults and pages are owned through `vaults.user_id`. After a valid access token, the Worker checks vault/page ownership before mutating data.

API routes

Implemented in `worker/index.ts`. Auth routes are public; every other `/api/*` path requires a valid Bearer JWT.

Method	Path	Purpose
POST	<code>/api/auth/signup</code>	Create user, set refresh cookie, return JWT + user
POST	<code>/api/auth/login</code>	Verify password, same tokens as signup
POST	<code>/api/auth/refresh</code>	Rotate refresh cookie, return new JWT
POST	<code>/api/auth/logout</code>	Delete refresh row, clear cookie
GET	<code>/api/auth/me</code>	Current user from Bearer JWT
GET	<code>/api/auth/google</code>	Redirect to Google OAuth consent
GET	<code>/api/auth/google/callback</code>	Exchange code, set cookie, redirect to <code>/</code>
POST	<code>/api/liveblocks-auth</code>	Mint Liveblocks session (requires app JWT first)

Signup rules

- Email must match `/^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/`
- Password minimum length: **8**
- Duplicate email → `409` "An account with this email already exists"
- Success → `201` `{ user, accessToken }` plus `Set-Cookie: vh_refresh=...`

Login rules

- Email is trimmed and lowercased
- Unknown user or bad password → 401 "Invalid email or password" (same message either way)

Refresh

1. Read `vh_refresh` cookie.
2. Hash it, look up unexpired row, get `user_id`.
3. Delete the old row.
4. Insert a new refresh token and set a new cookie.
5. Return `{ accessToken }`.

Logout

Revokes the current refresh token (if present) and sets the cookie `Max-Age=0`.

Client flow

Files

- `src/components/auth/AuthScreen.tsx` — login / signup UI and "Continue with Google"
- `src/store/useAuthStore.ts` — Zustand auth state
- `src/App.tsx` — boot `init()`, gate UI
- `src/main.tsx` — on persistent 401, call `logout()`
- `src/lib/api.ts` — authenticated `fetch` with refresh retry
- `src/components/navbar/Navbar.tsx` — shows email and Log out

Boot (`init`)

On mount, `App` calls `useAuthStore.init()`:

1. `POST /api/auth/refresh` with cookies.
2. If that fails, status is `signedOut` (Auth screen).
3. If it succeeds, store the new JWT in `localStorage`.
4. `GET /api/auth/me` with `Authorization: Bearer`.
5. Status becomes `signedIn` with `{ id, email, name }`.
6. Then `useVaultStore.loadVaults()` runs.

Statuses: `checking` → `signedIn` | `signedOut` | `loggingIn`.

Email login / signup

`AuthScreen` posts JSON via the store (`credentials: "include"` so the refresh cookie is stored). On success it writes `vh_access_token` and sets `signedIn`.

Silent 401 handling

`api()` in `src/lib/api.ts`:

1. Sends the current access token.
2. On `401` (except paths under `/api/auth/`), calls `/api/auth/refresh`.
3. Retries the original request with the new JWT.
4. If still `401`, runs `setUnauthorizedHandler` → `logout()`.

Logout

Clears `vh_access_token` / `vh_refresh_token` from `localStorage`, resets Zustand, then `POST /api/auth/logout` to revoke the cookie server-side.

Google OAuth

Start

The Auth screen navigates the browser to `/api/auth/google`. The Worker redirects to:

```
https://accounts.google.com/o/oauth2/v2/auth
```

with:

- `client_id` = `GOOGLE_CLIENT_ID`
- `redirect_uri` = `{origin}/api/auth/google/callback`
- `response_type` = `code`
- `scope` = `openid email profile`
- `access_type` = `online`
- `prompt` = `select_account`

Callback

1. Exchange `code` at `https://oauth2.googleapis.com/token`.
2. Load profile from `https://www.googleapis.com/oauth2/v3/userinfo`.
3. Find user by `google_sub` **or** matching email (password accounts can later sign in with Google).
4. Update `google_sub` and `avatar`, or insert a new user.
5. Create a refresh token, set `vh_refresh`, **302 redirect to** `/`.
6. The SPA boots, `init()` refreshes, and the user is signed in **without** a JWT in the Google callback body (JWT comes from `/api/auth/refresh`).

Required Worker secrets: `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`.

Protecting the rest of the API

After auth routes, the Worker:

```
const user = await getUserFromRequest(...)
if (!user && path.startsWith("/api/")) return 401 "Not signed in"
```

Vault and page handlers take `user.id` and check:

- `ownsVault(db, vaultId, userId)`
- `getPageWithVault(...)` (page joined to vault owned by the user)

A valid JWT for user A cannot read user B’s vaults.

Liveblocks (collaboration)

`POST /api/liveblocks-auth` sits **after** the JWT gate, so the browser must send a valid app access token.

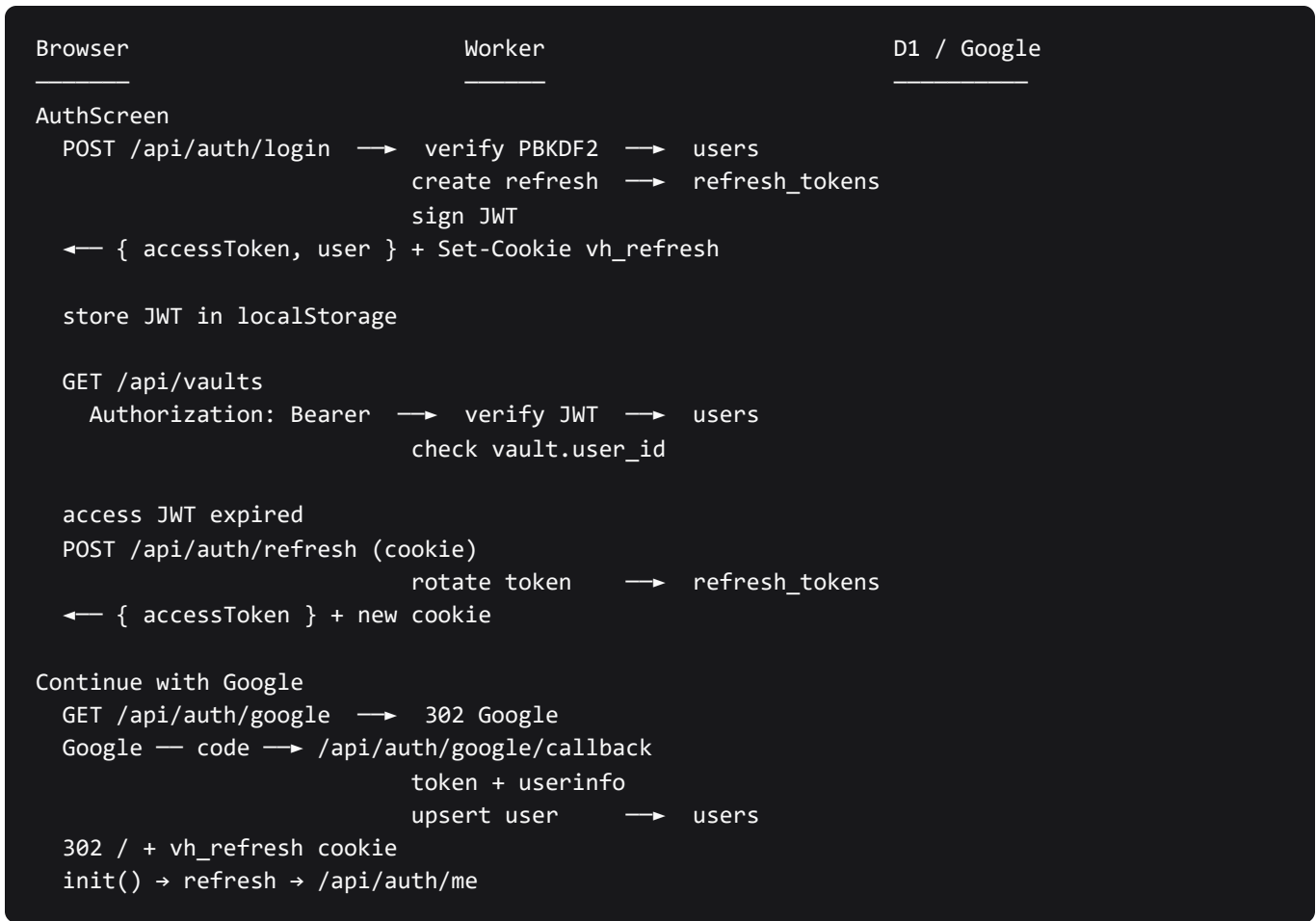
The handler then calls Liveblocks `https://api.liveblocks.io/v2/authorize-user` with `LIVEBLOCKS_SECRET_KEY`. Presence `userId` / `userName` currently come from the request body / `getOrCreateCollabUser()`, not necessarily the D1 user id.

A second client in `src/lib/liveblocksClient.ts` uses `authEndpoint: "/api/liveblocks-auth"` without attaching the JWT in that file; the primary editor path is `src/lib/liveblocks.ts`, which does send `Authorization`.

Environment (Worker)

Binding / secret	Use
DB	D1 database
JWT_SECRET	HMAC key for access JWTs
GOOGLE_CLIENT_ID	Google OAuth
GOOGLE_CLIENT_SECRET	Google OAuth
LIVEBLOCKS_SECRET_KEY	Collaboration auth
ASSETS	Static frontend (non-API requests)

Diagram



Source map

File	What it does
src/App.tsx	Auth gate and vault picker after sign-in
src/main.tsx	Global logout on unrecoverable 401
src/store/useAuthStore.ts	Client session
src/lib/api.ts	Bearer + refresh retry
src/components/auth/AuthScreen.tsx	UI
src/components/navbar/Navbar.tsx	Logout
worker/index.ts	All server auth
migrations/0001_init.sql	Users (and unused sessions)
migrations/0002_refresh_tokens.sql	Refresh rows

File	What it does
migrations/0003_google_auth.sql	google_sub , avatar